

Data Structures and Algorithms · Course Syllabus

Course Information

- **Subject**: Data Structures and Algorithms
- **Duration**: 1 Semester
- **Credits**: 4

Unit 1: Introduction to Data Structures

1.1 Arrays

An array is a linear data structure that stores elements of the same type in contiguous memory locations. Arrays provide $O(1)$ access.

Types of arrays:

- One-dimensional arrays: A simple list of elements
- Two-dimensional arrays (matrices): Rows and columns for tabular data
- Dynamic arrays: Arrays that can grow in size (e.g., ArrayList in Java, list in Python)

Time Complexity:

- Access: $O(1)$
- Search: $O(n)$ for unsorted, $O(\log n)$ for sorted (binary search)
- Insertion: $O(n)$ worst case (shifting elements)
- Deletion: $O(n)$ worst case (shifting elements)

1.2 Linked Lists

A linked list is a linear data structure where elements (nodes) are connected via pointers. Unlike arrays, linked lists do not require contiguous memory.

Types of linked lists:

- Singly Linked List: Each node points to the next node
- Doubly Linked List: Each node points to both next and previous nodes
- Circular Linked List: The last node points back to the first node

Operations:

- Insertion at beginning: $O(1)$
- Insertion at end: $O(n)$ for singly, $O(1)$ with tail pointer
- Deletion: $O(n)$ for searching + $O(1)$ for unlinking
- Traversal: $O(n)$

Advantages over arrays:

- Dynamic size · no need to declare size upfront
- Efficient insertion/deletion at the beginning
- No memory wastage (no pre-allocation)

Unit 2: Stacks and Queues

2.1 Stacks

A stack is a LIFO (Last In, First Out) data structure. The last element added is the first to be removed. Think of it like a stack of plates.

Operations:

- push(item): Add an item to the top · $O(1)$
- pop(): Remove and return the top item · $O(1)$
- peek()/top(): Return the top item without removing · $O(1)$
- isEmpty(): Check if the stack is empty · $O(1)$

Applications:

- Expression evaluation and conversion (infix to postfix)
- Undo/redo functionality in editors
- Function call management (call stack)
- Backtracking algorithms (maze solving, N-Queens)
- Browser history (back button)

****Implementation:**** Can be implemented using arrays or linked lists.

2.2 Queues

A queue is a FIFO (First In, First Out) data structure. The first element added is the first to be removed. Like a queue of people waiting for a bus.

Operations:

- enqueue(item): Add an item to the rear · $O(1)$
- dequeue(): Remove and return the front item · $O(1)$
- front(): Return the front item without removing · $O(1)$
- isEmpty(): Check if the queue is empty · $O(1)$

Types of queues:

- Simple Queue: Basic FIFO
- Circular Queue: The rear wraps around to the front, avoiding wasted space
- Priority Queue: Elements have priority, highest priority dequeued first
- Deque (Double-Ended Queue): Insertion and deletion from both ends

Applications:

- CPU scheduling (round-robin)
- Print job spooling
- Breadth-First Search (BFS)
- Buffer management in streaming

Unit 3: Trees

3.1 Binary Trees

A binary tree is a hierarchical data structure where each node has at most two children (left and right).

Terminology:

- Root: The topmost node
- Leaf: A node with no children
- Height: The longest path from root to a leaf
- Depth: The distance from the root to a node

Types:

- Full Binary Tree: Every node has 0 or 2 children
- Complete Binary Tree: All levels filled except possibly the last
- Perfect Binary Tree: All internal nodes have 2 children, all leaves at same level
- Balanced Binary Tree: Height difference between left and right subtrees is at most 1

3.2 Binary Search Trees (BST)

A BST is a binary tree where for each node: all left descendants < node < all right descendants.

Operations:

- Search: $O(\log n)$ average, $O(n)$ worst case (skewed tree)
- Insert: $O(\log n)$ average, $O(n)$ worst case
- Delete: $O(\log n)$ average · three cases (leaf, one child, two children)
- In-order traversal gives sorted output

3.3 Tree Traversals

- In-order (Left, Root, Right): Used to get sorted output from BST
- Pre-order (Root, Left, Right): Used to create a copy of the tree
- Post-order (Left, Right, Root): Used to delete the tree
- Level-order (BFS): Uses a queue, visits level by level

Unit 4: Sorting Algorithms

4.1 Comparison-Based Sorting

Bubble Sort:

- Repeatedly swap adjacent elements if they're in wrong order
- Time: $O(n^2)$ average and worst, $O(n)$ best (already sorted)
- Space: $O(1)$ · in-place
- Stable sort

Selection Sort:

- Find the minimum element and place it at the beginning
- Time: $O(n^2)$ always
- Space: $O(1)$ · in-place
- Not stable

Insertion Sort:

- Build sorted array one element at a time
- Time: $O(n^2)$ average and worst, $O(n)$ best
- Space: $O(1)$ · in-place
- Stable, efficient for small/nearly sorted data

Merge Sort:

- Divide the array in half, recursively sort each half, then merge
- Time: $O(n \log n)$ always
- Space: $O(n)$ · not in-place
- Stable sort
- Based on divide-and-conquer paradigm

Quick Sort:

- Choose a pivot, partition elements around it, recursively sort partitions
- Time: $O(n \log n)$ average, $O(n^2)$ worst case (bad pivot)
- Space: $O(\log n)$ · for recursion stack
- Not stable
- Usually fastest in practice

4.2 Time Complexity Comparison

Algorithm	Best	Average	Worst	Space	Stable?
Bubble	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	Yes
Selection	$O(n^2)$	$O(n^2)$	$O(n^2)$	$O(1)$	No
Insertion	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	Yes
Merge	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(n)$	Yes
Quick	$O(n \log n)$	$O(n \log n)$	$O(n^2)$	$O(\log n)$	No

Unit 5: Searching and Hashing

5.1 Searching Algorithms

****Linear Search:**** Check each element one by one. $O(n)$ time. Works on unsorted data.

****Binary Search:**** Requires sorted array. Compare with middle element, discard half. $O(\log n)$ time. Much faster for large datasets

5.2 Hashing

Hashing maps keys to array indices using a hash function for $O(1)$ average-case lookup.

****Hash Function:**** Maps a key to an index. A good hash function distributes keys uniformly.

Collision Resolution:

- Chaining: Each slot holds a linked list of elements that hash to the same index
- Open Addressing: Find another slot using probing
- Linear Probing: Check next slot sequentially
- Quadratic Probing: Check slots at quadratic intervals
- Double Hashing: Use a second hash function for step size

****Load Factor:**** ratio of filled slots to total slots. Rehashing occurs when load factor exceeds threshold (typically 0.75).

Unit 6: Graphs

6.1 Graph Representation

A graph $G = (V, E)$ consists of vertices V and edges E .

Types:

- Directed vs Undirected
- Weighted vs Unweighted
- Cyclic vs Acyclic

Representations:

- Adjacency Matrix: $O(V^2)$ space, $O(1)$ edge lookup
- Adjacency List: $O(V + E)$ space, $O(\text{degree})$ edge lookup

6.2 Graph Traversals

Breadth-First Search (BFS):

- Uses a queue
- Visits all neighbors before going deeper
- Time: $O(V + E)$
- Applications: shortest path (unweighted), level-order traversal

Depth-First Search (DFS):

- Uses a stack (or recursion)
- Goes as deep as possible before backtracking
- Time: $O(V + E)$
- Applications: cycle detection, topological sort, connected components

6.3 Shortest Path Algorithms

Dijkstra's Algorithm:

- Finds shortest path from a source to all vertices in a weighted graph
- Uses a priority queue (min-heap)
- Time: $O((V + E) \log V)$ with a binary heap
- Does NOT work with negative edge weights

Bellman-Ford Algorithm:

- Handles negative edge weights
- Time: $O(V * E)$

- Can detect negative weight cycles